Embedded Logic Analyzer using Raspberry Pi

Author: Electrical Engineering Graduate

Focus: Embedded Systems / VLSI / Digital Debugging

Unlocking Digital Insights: Project Overview and Goals

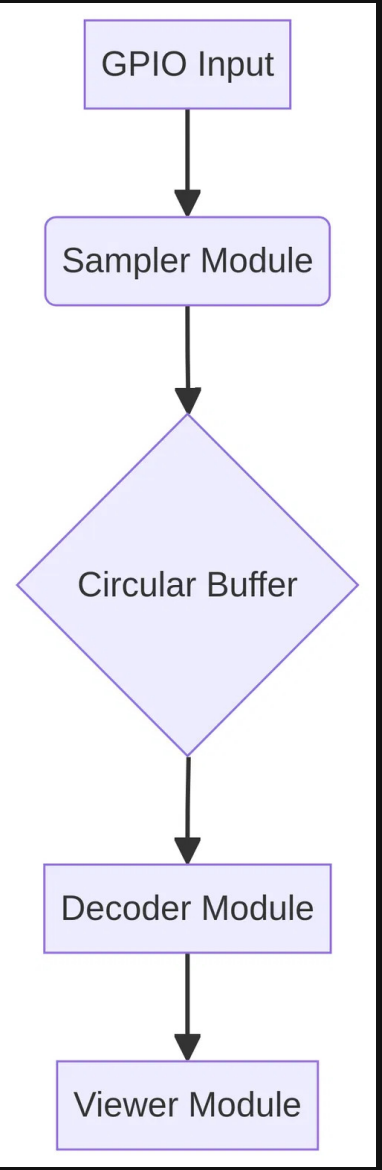
What is a Logic Analyzer?

- An electronic instrument that captures and displays multiple digital signals simultaneously.
- Essential for verifying digital designs and troubleshooting hardware-software interactions.
- Key Use Cases: Debugging microcontrollers, FPGAs, ASICs, and analyzing serial communication (UART, SPI, I2C).

Project Goals

- Achieve high-speed GPIO sampling on Raspberry Pi.
- Efficiently store captured waveform data.
- Implement robust digital protocol decoding (UART, SPI).
- Develop a clear waveform visualization tool.
- Design a modular and extensible software architecture.

From GPIO to Display: The Logic Analyzer Data Flow



- **GPIO Input:** Raw digital signals from the target device.
- **Sampler Module:** Captures high-speed edge transitions and timestamps.
- **Circular Buffer:** Stores sampled data efficiently in real-time.
- **Decoder Module:** Interprets raw data into meaningful protocol frames.
- **Viewer Module:** Presents decoded data as digital waveforms.

Raspberry Pi 4: The Heart of Our Logic Analyzer

TECHNICAL SPECIFICATIONS

Platform

Raspberry Pi 4 Model B (Broadcom BCM2711)

Sampling Method

DMA-based pigpio library for high-speed, low-latency capture.

Edge Detection

Interrupt-driven callbacks with microsecond-level precision.

Loopback Connection: A physical jumper wire connects Pin 11 (GPIO17) to Pin 12 (GPIO18) for internal signal verification.

GPIO CONFIGURATION

3.3V Power	1	2	5V Power
GPIO 2 (SDA)	3	4	5V Power
GPIO 3 (SCL)	5	6	Ground
GPIO 4	7	8	GPIO 14 (TX)
Ground	9	10	GPIO 15 (RX)
GPIO 17 (IN)	11	12	GPIO 18 (OUT)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23

Simplified 40-pin Header View (Pins 1-16)

Modular Design: A Scalable Software Foundation

main.py
Orchestrates application lifecycle and user interaction.

sampler.py
Interfaces with pigpio for microsecond edge capture.

buffer.py
Circular buffer (deque) for real-time data storage.

decoder_uart.py / decoder_spi.py
Protocol-specific logic for bitstream reconstruction.

viewer.py
Renders digital waveforms for visual timing analysis.

sampler.py implementation

```python
import pigpio

class Sampler:
    def __init__(self, pi, gpio_pin, callback):
        self.pi = pi
        self.gpio = gpio_pin
        self.cb_func = callback

        # Configure GPIO
        self.pi.set_mode(gpio_pin, pigpio.INPUT)

        # Set DMA-based callback
        self.cb = self.pi.callback(
            gpio_pin,
            pigpio.EITHER_EDGE,
            self._on_edge
        )

    def _on_edge(self, gpio, level, tick):
        # tick is microsecond timestamp
        self.cb_func(gpio, level, tick)
```

PRECISION CAPTURE: EDGE-TRIGGERED SAMPLING

Edge-Triggered Efficiency

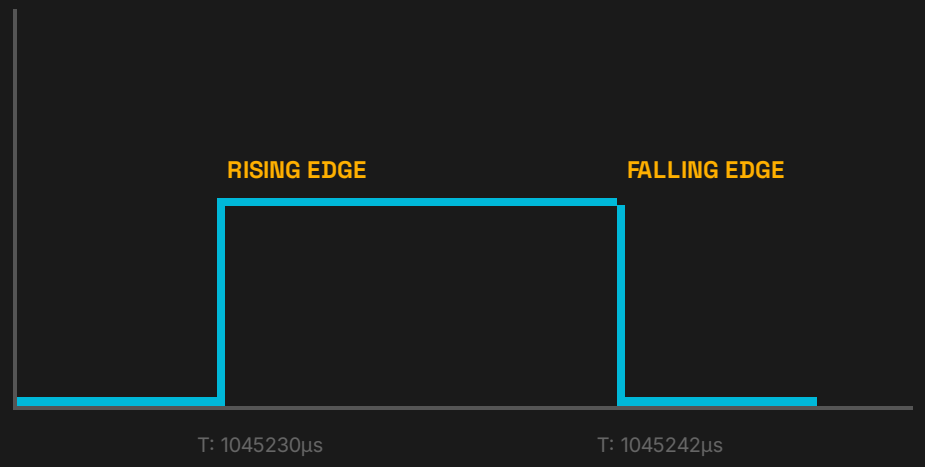
Captures data only on signal transitions (Rising/Falling), drastically reducing CPU overhead compared to continuous polling.

pigpio Callbacks

Utilizes DMA-based hardware timed callbacks. The pigpio daemon samples GPIO state independently of the Python process.

Microsecond Resolution

Each event is timestamped with a 32-bit 'tick' value, providing 1µs precision for accurate protocol reconstruction.



CONCEPTUAL TIMING RECONSTRUCTION

Real-time Data Handling: The Circular Buffer

Circular Buffer Implementation

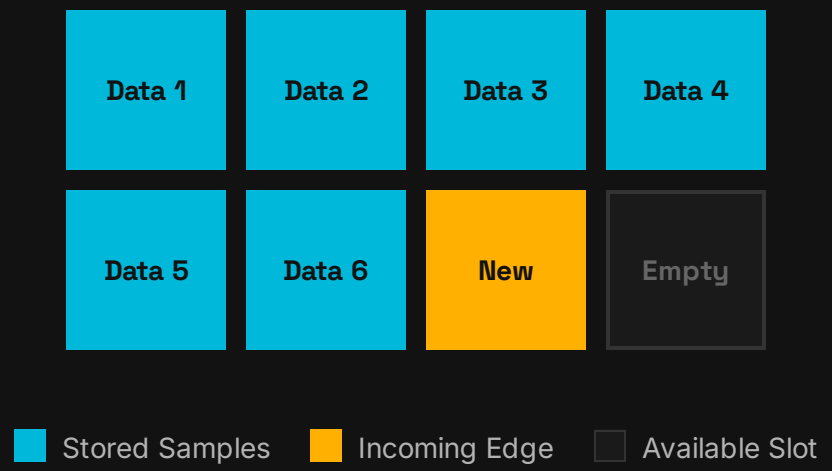
Utilizes Python's **collections.deque** with a fixed *maxlen*. This provides O(1) performance for appends and pops from either end, critical for high-frequency sampling.

FIFO Storage Logic

New samples automatically overwrite the oldest data once the buffer reaches capacity. This ensures the most recent signal transitions are always available for decoding.

Memory Optimization

Prevents unbounded memory growth during long-duration captures. The fixed-size allocation allows for predictable system resource usage on the Raspberry Pi.



Decoupling Sampling from Decoding

Making Sense of Bits: UART and SPI Protocol Decoding

UART Decoder

1. **Start Bit Detection**
   Identifies the high-to-low transition marking the beginning of a frame.
2. **Bit Timing**
   Calculates sampling points at the center of each bit period based on baud rate.
3. **Byte Reconstruction**
   Assembles 8 data bits (LSB first) and verifies parity/stop bits.

SPI Decoder

1. **Clock Synchronization**
   Monitors SCK edges to trigger data sampling (CPOL/CPHA dependent).
2. **MOSI Sampling**
   Captures the logic level of the Master Out Slave In line on active clock edges.
3. **Frame Reconstruction**
   Shifts bits into bytes and uses Chip Select (CS) to define frame boundaries.

Visualizing Signals: Waveform Viewer and Practical Test Setup

WAVEFORM VIEWER FEATURES

Digital Waveform Display

Renders captured and decoded signals as intuitive digital waveforms, showing logic levels over time.

Time vs. Logic Level

Clearly visualizes signal transitions, pulse widths, and timing relationships for precise analysis.

Transition Visualization

Highlights rising and falling edges, crucial for identifying glitches or timing violations.

Debugging Capability

Provides a visual representation that aids in quickly pinpointing issues in digital circuits.

PRACTICAL TEST SETUP

GPIO18 Output Signal

Configured to generate a test signal (e.g., a square wave or a simple UART/SPI sequence).

GPIO17 Sampling Input

Connected to capture the signal generated by GPIO18 for internal verification.

Loopback Connection

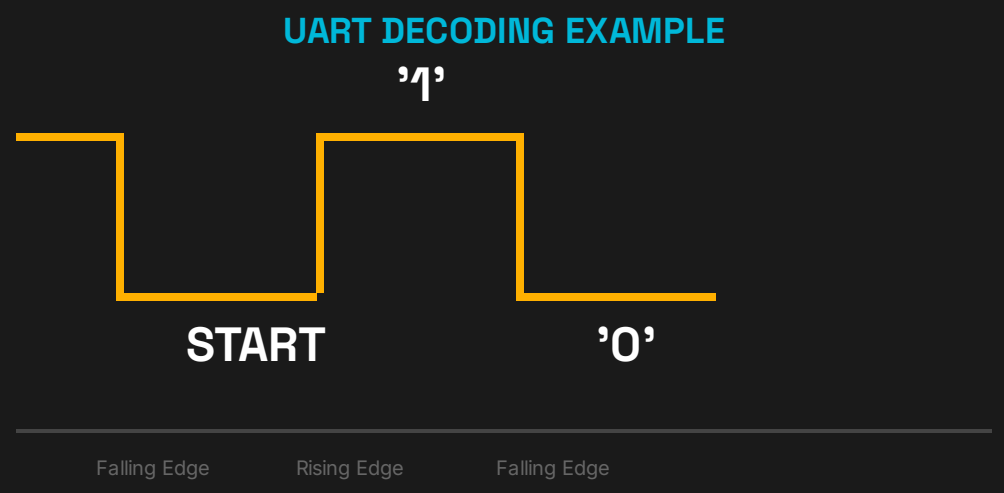
A physical jumper wire between GPIO17 and GPIO18 for self-testing and demonstration.

Real Signal Capture

Allows for immediate verification of the logic analyzer's functionality with a known signal.

LOOPBACK: GPIO18 (OUT) → JUMPER → GPIO17 (IN)

Decoding in Action: Example Waveform and Engineering Challenges



Reconstructing digital logic from microsecond timestamps.

Sampling Speed

Achieving microsecond-level accuracy on a non-real-time Linux OS. Solved using DMA-based hardware sampling via pigpio.

Timing Accuracy

Ensuring consistent timestamps across high-frequency transitions to prevent protocol decoding errors.

Buffer Overflow

Managing high-speed data bursts without losing edge transitions. Addressed with an optimized circular buffer.

Synchronization

Maintaining precise phase relationships between clock and data lines in synchronous protocols like SPI.

Optimizing Performance & Charting Future Directions

CURRENT PERFORMANCE

Sampling Resolution	1µs (Configurable)
Max Buffer Size	RAM Dependent
CPU Usage (Sampling)	< 5% (DMA-based)
Timing Accuracy	Microsecond-level
Data Latency	Minimal (Real-time)

FUTURE ROADMAP

GUI Waveform Viewer
Interactive interface for zooming and panning across captured signals.

Protocol Auto-detection
Heuristic analysis to identify UART, SPI, or I2C automatically.

Advanced Trigger System
Complex pattern matching and sequence-based acquisition triggers.

Multi-channel Sampling
Simultaneous capture across 8+ GPIO pins for parallel bus analysis.

Engineering Acumen: Skills Demonstrated and Project Conclusion

CORE COMPETENCIES

- Embedded Linux Development
- Low-Level GPIO Programming
- Digital Signal & Timing Analysis
- Real-time Data Acquisition
- Python Hardware Interfacing
- Systematic Hardware Debugging

PROJECT SUMMARY

- Successfully developed a functional and extensible Raspberry Pi-based logic analyzer.
- Demonstrated strong foundational knowledge in embedded systems and digital hardware.
- Applied practical problem-solving to overcome non-real-time OS sampling constraints.
- Created a valuable tool for digital hardware debugging and protocol analysis.

Thank you for your time and consideration.